Further simplification of Rocq and Lean proof interfaces

Proof simplification research

22 October 2026

1 What changed in the maintained proofs

The reviewed main pin is

`57541fe0cfe3af62b4cb58747d8d1d7c16b338d5`

It contains 26 commits after the preceding review. Phase 11 shortens the generic dominated-implication core to 287 lines and 13,723 bytes, with 56 private steps and three public modus-ponens bodies. It preserves its API and axiom-free generic statements. The larger shared unrestricted-basis adapter remains a separate proposal.

Phase 12 splits semantic imports into smaller model interfaces while preserving compatibility wrappers. The endpoint closure decreases from 192 to 179 modules, with 233,503 fewer selected source bytes. This is a dependency reduction, not a measured build speedup or deletion of the compatibility code. Its accepted primary execution covers 184 providers: 179 endpoint modules and five prefix modules. Fifteen are fresh and 169 reused with recorded source/import/object binding. The unchanged endpoint clients are fresh; ten named standard-axiom reports have no audit warnings. Separate compatibility checking covers 67 project modules and two clients, with five fresh compilations. The union of 199 distinct project modules is *not* a fresh 199-module build.

Independent inspection matches all 199 current source files to the accepted review hashes and all 167 archive members to its index. This establishes current-byte identity with recorded evidence, not local compilation or a fresh audit of every inherited object. The nine runtime batches retain their earlier 6,176-outcome evidence; phase 12 did not recompile them. No new Rocq check accompanies these phases.

Current Rocq source, enumeration and dominated-implication files are unchanged from the prior pin. In particular, its older offset source facade is not the historical normalized Rocq 9.1 endpoint input. This does not show the offset proof is incorrect; it prevents attributing the normalized receipt to different current bytes.

Proof Contracts already checks the semantic runtime interface and two decrement outcomes, not all 6,176. Its Rocq adapters have Coq 8.18 checks, not a production 9.1 endpoint rebuild. Semantic contracts, direct full-seed, shared implication and semantic bisimulation are prior proposals, not new discoveries here.

2 Use one quotient model implementation with checked adapters

The strongest new simplification is to promote the existing weaker `QHyp`-parameterized quotient development to the shared implementation, and instantiate it from the older `ModelsCPGIC` interface through one adapter. This is different from phase 12's import split, shared implication derivations, or the prior direct-seed proposal.

Six current KQ1 modules explicitly declare themselves generated forks: Model, Separation, Basic, Power, Replacement and Collect. Together they contain 805 lines and 34,291 bytes. This is a *gross development*

inventory, not 805 removable lines. A 54-line, 2,201-byte formula translation and relational-satisfaction block is exactly duplicated in the old and forked model modules. Owned static checks establish these facts without running the generator.

2.1 The hypothesis adapter already has its providers

The existing `QHyp` interface has nine fields: formula-indexed GIC; reflexivity, symmetry, transitivity and extensionality of the interpreted equivalence; raw profile pairs; interpreted union and power; and singleton-closed containers. Existing KQ1 proofs derive the quotient model and its set-theoretic properties from precisely this interface. None of these fields is weakened.

The stronger CP/GIC model supplies all nine through existing profile, composition, union, power and singleton theorems. A concrete staging adapter and four compatibility clients are included as uncompiled Lean source. The K lower endpoint must retain its direct tagged-domain QHyp provider. Forcing it through the stronger CP/GIC assumptions would be wrong: its tagged domain intentionally lacks the older literal-power-container premise. Only the legacy CP/GIC lane uses the new adapter.

2.2 Check definitional compatibility before deleting anything

Both model-setoid definitions have literally the same interpreted equivalence relation. Their remaining fields are proofs in Prop. Their domains are quotients of that setoid; their constructors use the same quotient constructor; and their quotient membership is a lift of the identical predicate. After unfolding, proof irrelevance should identify the differing proof arguments.

The included audit therefore requires reflexivity proofs for equality of four old and shared objects: the setoid, quotient domain, constructor and membership relation. This is a strong source-level reason to expect definitional equality, not a compiled result. Compile these four clients first. If one fails, find the changed definition rather than silently insert casts or assume public signatures survived. This does not remove representative choices elsewhere, the later model construction, or classical and quotient assumptions from the endpoint.

2.3 Resolve the import cycle before making compatibility wrappers

The staging adapter currently imports old union and power providers. This is acyclic while both implementations remain. Replacing those providers by wrappers around the adapter would immediately create a cycle. The migration must instead proceed as follows:

1. Extract the common formula translation and satisfaction block into a neutral syntax module, preserving old names with wrappers.

2. Extract raw CP/GIC resource providers for interpreted union, power and singleton closure. Power also needs its trace and formula-only separation support. That support depends on syntax, not the instantiated quotient Separation theorem; do not import the old separation module wholesale.

3. Build the CP/GIC-to-QHyp adapter from these raw providers, profile/composition semantics and the neutral syntax.

4. Retain the existing KQ1 implementation as the common core. Make the old quotient modules thin compatibility wrappers around its CP/GIC instance, after the four equality checks pass.

5. Change the generation policy so it no longer recreates separate proof copies. Preserve historical records and rebuild unchanged endpoint and compatibility clients.

The common core and the adapter have different roles: the former proves the model theorem once under QHyp, while the latter discharges those exact hypotheses for a particular older model. Neither language is asked to trust the other's kernel.

The preliminary import/name patch removes **zero modules**: another dependency path retains the old implementation. Its narrower root closure stays at 167, not phase 12's 179-module endpoint/client scope. Neither this patch nor the gross 805-line inventory establishes net savings.

The inspected Rocq lower chain already reuses its generic interpreted proof predicate and standard lower construction rather than mirroring these generated Lean files. No matching 805-line Rocq deletion is proposed. Nor should Lean's proof-irrelevance-based reflexivity argument be copied into Rocq as an assumption: reuse identical setoid instances or prove explicit transport under its existing assumptions, without adding a proof-irrelevance axiom. The practical cross-language policy is to retain each language's reusable semantic core and add small checked adapters, not force identical file layouts.

3 A small refinement of the existing certificate adapter proposal

The current literal and normalized Lean certificate front ends duplicate an exact seven-line entry-check definition and seventeen-line soundness proof body. A supplied uncompiled patch factors their endpoint comparison through one entry dispatcher and its soundness lemma. Once wrappers and abstraction overhead are counted, it removes only six lines overall.

This is a concrete refinement of the preceding round's adapter-sharing idea, not a new conceptual discovery or substantial measured improvement. Keep the already shared semantic bisimulation theorem; the audit found no reason to generalize its single halting proof further. For Rocq, the previously proposed semantic adapter remains useful, but it must not be relabeled a new result here.

4 Opaque prefix contracts for control primitives

The new runtime experiment goes beyond replaying every residual leaf separately. Factor a consumed program counter as $P +\!\!+ S$, with literal suffix S and opaque prefix P. Require P to end in one and contain no run of eight zeros. Ordinary symbolic rules may not inspect any cell of P; only the Home scanner may traverse it through a new generic lemma. The complete prefix is retained, with its orientation explicit, rather than replaced by arbitrary bits.

On the exact formal K215 table and source, 1,421 of 1,425 control guards fit this abstraction: 1,325 continue guards and 96 break guards. They group into 76 template candidates, comprising 69 continue templates and seven break templates. Four control guards remain literal:

```
001/CB1   010111/CC3   1000111/CC3   11011/CB1
```

All 1,900 increment guards, 2,850 decrement branches and the single halt guard also remain literal. Thus the target is 76 template checks plus 4,755 literal checks, or 4,831 instead of 6,176. These are expensive template/literal simulation obligations; all 6,176 cheap source-coverage instances remain. The reduction of 1,345 simulation obligations, about 21.8%, is conditional on the new semantic framing proof. It is not a compiled coverage theorem or measured build-speed improvement. Failure to find an opaque abstraction is not divergence or an impossibility result; the literal obligation is retained.

4.1 The boundary condition that makes the Home argument work

Home scans left across the reversed prefix. The first prefix bit it sees is one, resetting its zero-run state regardless of the explicit suffix's incoming count from zero through seven. The prefix's no-eight-zero

invariant then prevents the forbidden eighth-zero transition internally. Finally the left guard one resets the state before the existing boundary return theorem. Merely forbidding eight zeros inside P, without requiring it to end in one, would leave a possible crossing run unchecked.

The intended contract quantifies over every finite prefix satisfying that invariant, all valid register/spare lists, and all gaps $g \geq 16$. It retains the standard blank-left dispatcher frame; it does *not* allow arbitrary nonblank tape beyond the left guard. It starts at the exact routed primitive state with the full prefix/suffix zipper and finishes at the exact dispatch frame for $P +\!\!+ S'$, with the stated gap change and unchanged registers. Residual execution starts with its progress flag false; positive progress must be established, not borrowed from a possibly zero-duration prefix.

Each of the 76 templates also has an independently checked source-operation framing condition. For continue by k, the literal suffix contains at least k ones and stripping them yields exactly S'. For break by k, it contains enough zeros, stripping remains inside the suffix, and the following increment does not overflow. Therefore these source operations ignore *every* admissible P, not just the observed finite prefixes. Gap change depends only on the suffix-length difference. Current source paths have length at most 48, so their maintained gap parameter instantiates $g \geq 16$ with the nonnegative shift $n' = n + 48 - |PC|$.

Eight negative boundary tests reject malformed prefixes, ordinary opaque reads, misplaced opaque Home chunks, crossing strip/carry and wrong target suffixes. These tests check the implemented admission rules, not universal soundness.

The remaining proof work is explicit: compile the Home crossing lemma using existing `scanBound_sound` and `home_word`; interpret an opaque segment in both orientations; extend interpreter soundness while rejecting ordinary reads of it; kernel-check the 76 runs and 4,755 literal obligations; and bind every source occurrence to its routed control state and prefix invariant. Feed these into the already checked semantic coverage interface. Rocq can reuse the finite data and word invariant, but must establish the contract in its own zipper semantics; no cross-kernel tape bridge is necessary.

5 Further research questions

Can raw-resource extraction and compatibility aliases produce a measured net source reduction after all adapters are counted? Can arithmetic-template families be shared without inspecting the opaque prefix? Which template partition minimizes combined kernel checking and coverage cost, rather than just the number of templates? These should be evaluated after the universal framing and compatibility proofs, not inferred from the current inventories.

6 Acceptance and reproduction

Compile the QHyp adapter and definitional-equality clients first, then perform the cycle-free extraction and compatibility migration. Rebuild unchanged endpoint and compatibility clients with the same assumption reports before measuring source and build costs. No Lean or Rocq compiler is available in this environment; received scripts and proof modules are inert evidence.

The packet includes pinned inputs, owned offline checks and explicit proof obligations. Run Python 3 with assertions enabled and a fresh external work directory:

```
python -I -B verify.py --work /tmp/round12-check
```

Replay checks pinned bytes, patch generation, model/import inventories and template assignments. It does not run received generators, compile proofs, or establish the new universal semantics. No smaller machine, faster build or reduced final axiom set is claimed.